

Un-Rippling: Recovering Scenes Seen Through Wavy Water Without Deep Learning

Digital Image Processing - Project Proposal

Problem Statement / Description

When you look at something through a moving water surface (the floor of a pool, a coin at the bottom of a fountain, a page under a tray of water), the image keeps wobbling. Every frame is bent differently by the waves, so text becomes unreadable and straight lines turn wavy. The goal is to take a short video of such a scene and recover a single sharp, undistorted image of what lies underneath.

This is a hard problem because the distortion is non-rigid, changes every frame, and is unknown. Existing work falls into two groups. Model-driven methods register the frames against each other and use the fact that water waves are smooth and roughly periodic [4, 5, 6]. Learning-based methods train a network on pairs of distorted and clean images [2, 3].

The interesting part is that deep learning does not clearly win here. James et al. [1] built a method with no training at all (feature tracking, sparse motion estimation in the Fourier domain and polynomial expansion optical flow) and it beat the deep network of Li et al. [2] on both synthetic and real videos. A 2026 benchmark [3] shows the same trend: at low wave amplitude, simple classical grid registration does better than their diffusion model on every metric and every wave type (for ocean waves, PSNR 25.44 vs 24.86 and LPIPS 0.071 vs 0.126), and on real lab videos the two are close. From medium amplitude upwards the diffusion model takes the lead, and at high amplitude the classical methods fall apart (for ocean waves, PSNR 15.1 vs 22.2), because multi-path refraction creates discontinuities and repeated copies of objects that a smooth warp cannot undo.

So the open question is how far a purely classical pipeline can be pushed before it breaks, and whether a few targeted additions can move that point further.

Proposed Solution

We will build a fully classical restoration pipeline in stages, measuring each stage on the same data so the effect of every addition is clear. The work is split into a minimum pipeline that we will definitely deliver, and stretch goals that we attempt once the minimum pipeline works end to end.

Minimum pipeline (guaranteed deliverable)

1. **Baselines.** Temporal mean and median of the frames. These give the blurry “long exposure” version of the scene and a first template.
2. **Iterative registration to a template.** Compute dense optical flow from each frame to the current template, warp every frame back, average them into a new template and repeat for a few rounds. This is the first stage of Oreifej et al. [4], using optical flow instead of B-splines.
3. **Low-rank baseline.** Split the stack of registered frames into a low-rank part (the static scene) and a sparse part (leftover glints and small misalignments) [8, 10], and average the low-rank part. This is the second stage of [4], so steps 2 and 3 together reproduce their method and form the classical baseline that step 4 must beat. We do not fuse its output, because the sparse part also

absorbs the frame-specific sharp detail that fusion needs.

4. **Our extension: lucky patch fusion after registration.** Even after registration, some frames are sharper than others in different regions, depending on how flat the water was at that spot and moment. We will split the registered frames from step 2 into overlapping patches, score each patch for sharpness (local gradient energy and similarity to the template), and blend the best patches with smooth weights, followed by mild sharpening. The idea of waiting for locally flat water goes back to Efros et al. [6], who pick the most typical patch at each location. Scoring patches by sharpness comes from lucky-region fusion for atmospheric and underwater turbulence [11, 12], which we adapt to registered frames.

Stretch goals

5. **Sparse motion estimation in the Fourier domain.** Track a set of strong corner points across the video. Water motion is smooth in space and roughly periodic in time, so the full motion field has a sparse representation in the 3D Fourier basis. We recover the motion at every pixel from the tracked points by solving an ℓ_1 minimisation problem, as done by James et al. [1], using our own FISTA solver [9]. This replaces or initialises step 2.

We will run every stage on videos sorted by distortion level, so we can show at which amplitude each method stops working.

Objectives

- Have the minimum pipeline (steps 1 to 4) working end to end by the end of week 3.
- Beat grid registration [3] at low and medium distortion and match the learning-based results of [3] at low distortion (PSNR, SSIM, LPIPS). At medium distortion, where their diffusion model leads by several dB, we report how much of the gap we close.
- Stretch: reimplement the method of James et al. [1] and check that our numbers are close to the reported ones.
- Show a clear improvement from lucky patch fusion over plain registration on our own captured videos.
- Keep runtime under about one minute per clip at 256×256 resolution on a normal laptop CPU (the diffusion model in [3] needed roughly a month of training on six GPUs).
- Find and explain the distortion level where the classical pipeline breaks down, with examples of what goes wrong.

Data

- Synthetic and real videos released with James et al. [1], plus the real videos of Tian and Narasimhan [5] (text seen through water, also used by [1]).
- The benchmark from [3]. Its code (GitHub) and data (Zenodo) generate synthetic sequences with four wave types at four amplitudes. Its real lab recordings do not appear to be public.

- Our own videos: a phone above a clear tray of water placed over printed text and checkerboards. We first record the still scene as ground truth, then disturb the water and record again.

Implementation Plan (what we write vs. what we use)

Everything will be written in Python. The table below lists which parts we implement ourselves and which parts come from existing libraries or code.

Component	Source
Mean and median baselines	From scratch (NumPy)
Iterative template registration loop	From scratch. Uses OpenCV for the optical flow itself (Farnebäck method [7], <code>cv2.calcOpticalFlowFarneback</code>) and for warping (<code>cv2.remap</code>)
Robust PCA baseline	From scratch (inexact augmented Lagrangian method [10])
Lucky patch fusion (sharpness scores, patch selection, blending)	From scratch
Point tracking (stretch)	OpenCV corner detection and KLT tracking (<code>goodFeaturesToTrack</code> , <code>calcOpticalFlowPyrLK</code>)
Sparse motion field recovery in the 3D Fourier basis (stretch)	From scratch. Our own FISTA solver, with NumPy FFT for the transforms
PSNR and SSIM	scikit-image
LPIPS	The official <code>lpips</code> package. It contains a pretrained network, but we use it only to score results so we can compare with [3], never inside our method
Results of learning-based methods (DATUM, V-cache, Li et al.)	Taken from the numbers reported in [1] and [3]. We will not retrain or run these models
Reference code of James et al. (MATLAB)	Used only to produce reference results on our own videos and to check our reimplementation

Timeline

Week	Work
1	Collect datasets, record our own videos, mean and median baselines, evaluation scripts
2	Iterative template registration
3	Robust PCA baseline and lucky patch fusion. Minimum pipeline complete
4	Experiments across distortion levels. Stretch: sparse Fourier motion recovery [1]
5	Final comparisons, runtime measurements, failure analysis, report and demo videos

Deliverables

1. Our implementation of the minimum pipeline (baselines, iterative registration, the robust PCA baseline and lucky patch fusion), plus sparse Fourier motion recovery if the stretch goal is reached, with the split between our code and library code as listed above.
2. A small dataset of our own distorted and clean video pairs.
3. Result tables (PSNR, SSIM, LPIPS and runtime) on the data from [1], the benchmark from [3] and our own videos, grouped by distortion level, next to the reported numbers of the learning-based methods.
4. A short analysis of where the classical approach works and where it fails, with before and after images and videos.

References

- [1] J. G. James, P. Agrawal, and A. Rajwade. Restoration of non-rigidly distorted underwater images using a combination of compressive sensing and local polynomial image representations. In *ICCV*, 2019.
- [2] Z. Li, Z. Murez, D. Kriegman, R. Ramamoorthi, and M. Chandraker. Learning to see through turbulent water. In *WACV*, 2018.
- [3] M. V. Shugaev et al. A unified benchmark for multi-frame image restoration under severe refractive warping. In *CVPR Workshops (MaCVi)*, 2026. arXiv:2605.05079.
- [4] O. Oreifej, G. Shu, T. Pace, and M. Shah. A two-stage reconstruction approach for seeing through water. In *CVPR*, pp. 1153–1160, 2011.
- [5] Y. Tian and S. G. Narasimhan. Seeing through water: Image restoration using model-based tracking. In *ICCV*, pp. 2303–2310, 2009.
- [6] A. Efros, V. Isler, J. Shi, and M. Visontai. Seeing through water. In *NIPS*, pp. 393–400, 2004.
- [7] G. Farnebäck. Two-frame motion estimation based on polynomial expansion. In *SCIA*, LNCS 2749, pp. 363–370, 2003.
- [8] E. J. Candès, X. Li, Y. Ma, and J. Wright. Robust principal component analysis? *Journal of the ACM*, 58(3):1–37, 2011.
- [9] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [10] Z. Lin, M. Chen, and Y. Ma. The augmented Lagrange multiplier method for exact recovery of corrupted low-rank matrices. *arXiv:1009.5055*, 2010.
- [11] M. Aubailly, M. A. Vorontsov, G. W. Carhart, and M. T. Valley. Automated video enhancement from a stream of atmospherically-distorted images: the lucky-region fusion approach. In *Proc. SPIE* 7463, 74630C, 2009.
- [12] A. V. Kanaev, W. Hou, S. R. Restaino, S. Matt, and S. Gładysz. Restoration of images degraded by underwater turbulence using structure tensor oriented image quality (STOIQ) metric. *Optics Express*, 23(13):17077, 2015.